# Day 63 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 1 reference

Watch: Long Short-Term Memory (LSTM), Clearly Explained (StatQuest with Josh Starmer) | Illustrated Guide to LSTM's and GRU's: A step by step explanation (The AI Hacker)

`day63_lstm_cell_gates.py`, segment by segment, in the order `main()` uses things. The code boxes are cut from the real script by program (inline comments removed so lines fit). Where a chunk implements a formula, a purple **MATH** box shows the formula first. The green **SYSTEM DIAGRAM** shows how the cell's pieces connect, and the graphs are the ones the script itself saved. A glossary of new or reinforced syntax is at the end.

**MATH BOX LEGEND**

**Symbols used in the MATH boxes:** x = this step's input · h = hidden state · c = cell state · f, i, o = forget, input and output gates · g = candidate · W, b = weights and biases · a dot between two symbols = multiply entry by entry

## Segment 0 — Imports and global setup

### Imports

**CODE**

```
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, Circle
np.random.seed(42)
```

- `import time` — gives **time.perf_counter()**, a precise clock used in Segment 6 to time the two cells.
- `matplotlib.use("Agg")` — draw to image files with no window; must come before importing pyplot.
- `from matplotlib.patches import FancyBboxPatch, Circle` — two shapes for the cell diagram: a rounded rectangle and a circle.
- `np.random.seed(42)` — fixes NumPy's global random numbers; every experiment below also uses its own seeded generator.

## Segment 1 — Helpers

### Three helpers: two reused, one new

**MATH**

$$sigmoid(z) = 1 / (1 + e^{-z}) \quad z \text{ very negative} \to 0, \quad z = 0 \to 0.5, \quad z \text{ very positive} \to 1$$

**CODE**

```
def rnn_step_forward(x_t, h_prev, Wxh, Whh, bh):
    return np.tanh(x_t @ Wxh + h_prev @ Whh + bh)
def spectral_radius(W):
    return np.max(np.abs(np.linalg.eigvals(W)))
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
```

- `rnn_step_forward / spectral_radius` — copied unchanged from Days 57 and 59. They are here so the LSTM can be compared with the plain RNN on equal terms.

- `np.linalg.eigvals(W)` — the eigenvalues of a square matrix (they may be complex numbers).
- `np.max(np.abs(...))` — the largest size among them: the spectral radius.
- `np.exp(-z)` — e to the power of minus z, for every entry of z at once.
- `1.0 / (1.0 + np.exp(-z))` — the sigmoid. Writing 1.0 instead of 1 makes clear the result is a decimal number.

## Segment 2 — The LSTM cell

### lstm_step_forward

**MATH**

$$f = sigmoid( [x, h] W_f + b_f ), \quad i = sigmoid( [x, h] W_i + b_i )$$

$$g = tanh( [x, h] W_g + b_g ), \quad o = sigmoid( [x, h] W_o + b_o )$$

$$c_{new} = f \cdot c_{old} + i \cdot g, \quad h_{new} = o \cdot tanh( c_{new} )$$

**CODE**

```
def lstm_step_forward(x_t, h_prev, c_prev, W, b):
    H = h_prev.shape[1]
    a = np.concatenate([x_t, h_prev], axis=1) @ W + b
    f = sigmoid(a[:, 0 * H:1 * H])
    i = sigmoid(a[:, 1 * H:2 * H])
    g = np.tanh(a[:, 2 * H:3 * H])
    o = sigmoid(a[:, 3 * H:4 * H])
    c = f * c_prev + i * g
    h = o * np.tanh(c)
    return h, c, (f, i, g, o)
```

- `h_prev.shape[1]` — the number of columns of the hidden state: the hidden size H.
- `np.concatenate([x_t, h_prev], axis=1)` — glue the two arrays together left to right. (m, D) and (m, H) become (m, D + H).
- `... @ W + b` — one matrix product gives all four raw scores: W is (D + H, 4H), so the result is (m, 4H).
- `a[:, 0 * H:1 * H]` — columns 0 up to H: the forget gate's block. Writing 0 * H and 1 * H keeps the four lines looking alike.
- `a[:, 1 * H:2 * H]` — the next H columns: the input gate's block; then the candidate's, then the output gate's.
- `f * c_prev + i * g` — ***** between arrays multiplies entry by entry (not a matrix product).
- `o * np.tanh(c)` — squash the new cell state, then let the output gate decide how much of it to show.
- `return h, c, (f, i, g, o)` — three things come back: two arrays and a tuple of four arrays.

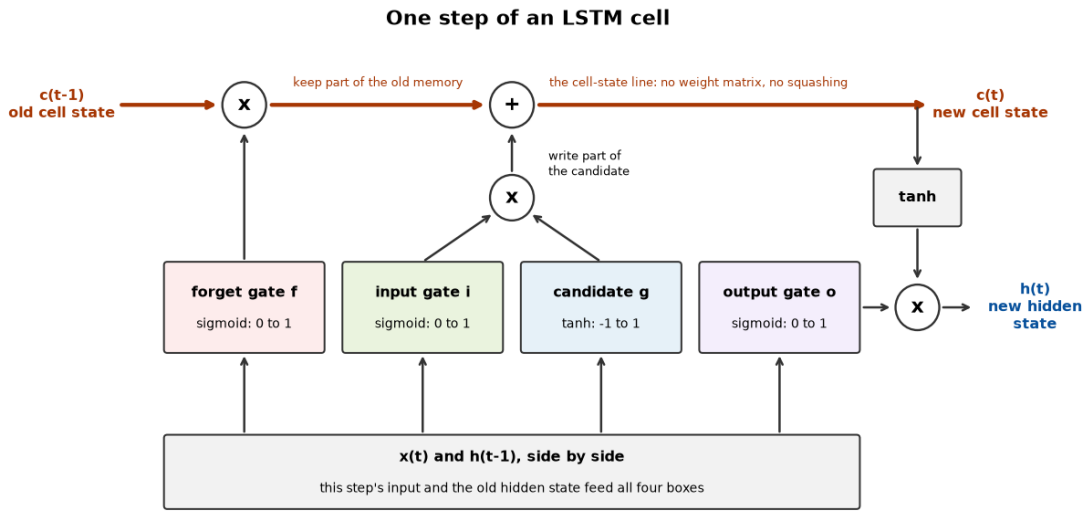
## init_lstm and cell_update

**CODE**

```
def init_lstm(input_dim, hidden_dim, seed, forget_bias=0.0, weight_scale=1.0):
    rng = np.random.RandomState(seed)
    W = rng.randn(input_dim + hidden_dim, 4 * hidden_dim) * weight_scale / np.sqrt(input_dim +
        hidden_dim)
    b = np.zeros(4 * hidden_dim)
    b[:hidden_dim] = forget_bias
    return W, b
def cell_update(c_prev, f, i, g, o):
    c = f * c_prev + i * g
    return c, o * np.tanh(c)
```

- `forget_bias=0.0, weight_scale=1.0` — two arguments with default values; callers name them only when they want something else.
- `rng.randn(input_dim + hidden_dim, 4 * hidden_dim)` — random numbers with the shape of the stacked weight matrix.
- `* weight_scale / np.sqrt(input_dim + hidden_dim)` — shrink the random numbers by the square root of the number of inputs to each unit.
- `b[:hidden_dim] = forget_bias` — the first H biases belong to the forget gate; set all of them at once.
- `return c, o * np.tanh(c)` — cell_update returns two values; the second is computed right in the return line.

**SYSTEM DIAGRAM**




*lstm_step_forward as a picture. The shared grey box at the bottom is np.concatenate; the four coloured boxes are the four slices of a; the circles are the two update lines.*

## Segment 3 — demo_one_step

### One step traced by hand

CODE

```
x, h_prev, c_prev = 1.0, 0.5, -0.8
W = np.array([[0.5, -1.0, 2.0, 1.5],
              [1.0, 0.4, -0.6, 0.2]])
b = np.array([0.1, 0.0, 0.0, -0.3])
a_f = x * W[0, 0] + h_prev * W[1, 0] + b[0]
a_i = x * W[0, 1] + h_prev * W[1, 1] + b[1]
a_g = x * W[0, 2] + h_prev * W[1, 2] + b[2]
a_o = x * W[0, 3] + h_prev * W[1, 3] + b[3]
f, i, g, o = sigmoid(a_f), sigmoid(a_i), np.tanh(a_g), sigmoid(a_o)
c = f * c_prev + i * g
h = o * np.tanh(c)
h_fn, c_fn, (f_fn, i_fn, g_fn, o_fn) = lstm_step_forward(np.array([[x]]), np.array([[h_prev]]),
    np.array([[c_prev]]), W, b)
```

- `x, h_prev, c_prev = 1.0, 0.5, -0.8` — three plain numbers assigned in one line.
- `np.array([[...], [...]])` — a 2-by-4 array written out by hand: row 0 is the weights from x, row 1 the weights from h.
- `x * W[0, 0] + h_prev * W[1, 0] + b[0]` — ordinary arithmetic on single numbers: the forget gate's raw score.
- `f, i, g, o = sigmoid(a_f), sigmoid(a_i), np.tanh(a_g), sigmoid(a_o)` — four values computed and unpacked in one line.
- `np.array([[x]])` — the function expects 2-D arrays (batch, size), so the single number is wrapped in two pairs of brackets: shape (1, 1).
- `h_fn, c_fn, (f_fn, i_fn, g_fn, o_fn) = ...` — nested unpacking: the third returned item is itself a tuple of four.

### Printing the comparison, and the random check

CODE

```
for name, raw, hand, fn in [("forget gate f (sigmoid)", a_f, f, f_fn), ("input gate i (sigmoid)",
    a_i, i, i_fn),
                            ("candidate g (tanh)", a_g, g, g_fn), ("output gate o (sigmoid)", a_o,
                                o, o_fn)]:
    print(f"{name:<26}{raw:>11.4f}{hand:>11.4f}{fn.item():>11.4f}")
Wx, Wh = W[:D], W[D:]
gate = lambda k: x_t @ Wx[:, k * H:(k + 1) * H] + h0 @ Wh[:, k * H:(k + 1) * H] + b[k * H:(k + 1)
    * H]
c_ref = sigmoid(gate(0)) * c0 + sigmoid(gate(1)) * np.tanh(gate(2))
h_ref = sigmoid(gate(3)) * np.tanh(c_ref)
```

- `for name, raw, hand, fn in [...]:` — each item of the list is a 4-item tuple, unpacked into four names.
- `{name:<26}` — format the text left-aligned in 26 characters; **>11.4f** means right-aligned, 11 wide, 4 decimals.
- `fn.item()` — turn a 1-by-1 array into a plain Python number so it can be formatted.
- `W[:D], W[D:]` — the first D rows (they multiply x) and the remaining rows (they multiply h).
- `gate = lambda k: ...` — a small one-line function stored in a name. gate(0) gives the forget gate's raw scores, gate(1) the input gate's, and so on.
- `Wx[:, k * H:(k + 1) * H]` — the k-th block of H columns.
- `sigmoid(gate(0)) * c0 + sigmoid(gate(1)) * np.tanh(gate(2))` — the textbook formula for the new cell state, one gate at a time.

## Segment 4 — demo_gates

### Setting the gates directly

**CODE**

```
c_prev, g = 0.80, -0.50
print(f"\n{'what the gates say':<34}{'f':>5}{'i':>5}{'o':>5}{'c_new':>9}{'h_new':>9}")
for name, f, i, o in [("keep the memory, show it", 1, 0, 1), ("keep the memory, hide it", 1, 0,
    0),
                      ("erase the memory", 0, 0, 1), ("replace it with the candidate", 0, 1, 1),
                      ("add the candidate to it", 1, 1, 1), ("half of each", 0.5, 0.5, 1)]:
    c, h = cell_update(c_prev, f, i, g, o)
    print(f"{name:<34}{f:>5}{i:>5}{o:>5}{c:>9.3f}{h:>9.3f}")
```

- `("keep the memory, show it", 1, 0, 1)` — a tuple holding a label and three gate values.
- `for name, f, i, o in [...]:` — loop over six such tuples.
- `c, h = cell_update(c_prev, f, i, g, o)` — the same two update lines as the real cell, with the gates given instead of computed.
- `{f:>5}` — no format type, so 1 prints as 1 and 0.5 as 0.5.

## Segment 5 — demo_store_hold_read

### handset_memory_cell

**MATH**

*raw score = ± k/2 → gate = sigmoid(± k/2)  k = 10: 0.9933 / 0.0067,  k = 40: 0.999999998 / 0.000000002*

**CODE**

```
def handset_memory_cell(k):
    W = np.zeros((4, 4))
    b = np.zeros(4)
    W[1, 0], b[0] = -k, k / 2
    W[1, 1], b[1] = k, -k / 2
    W[0, 2] = 1.0
    W[2, 3], b[3] = k, -k / 2
    return W, b
```

- `np.zeros((4, 4))` — 4 rows (value, write flag, read flag, old h) by 4 columns (f, i, g, o); all weights start at zero.
- `W[1, 0], b[0] = -k, k / 2` — two assignments in one line: the write flag's weight to the forget gate, and the forget gate's bias.
- `W[1, 1], b[1] = k, -k / 2` — the mirror image for the input gate: shut normally, open when write = 1.
- `W[0, 2] = 1.0` — the value feeds the candidate with weight 1.
- `row 3 stays zero` — the old hidden state is connected to nothing, so this cell's gates depend on the flags alone.

### Building the sequences and running the cell

**CODE**

```
n = 1000
rng = np.random.RandomState(0)
v = rng.uniform(-1, 1, n)
noise = rng.randn(n, 999) * 0.5
stored = np.tanh(v)
X = np.zeros((n, T, 3))
X[:, 0, 0], X[:, 0, 1] = v, 1.0
X[:, 1:, 0] = noise[:, :T - 1]
X[:, -1, 2] = 1.0
h, c = np.zeros((n, 1)), np.zeros((n, 1))
hidden_peak, trace = 0.0, []
for t in range(T):
    h, c, _ = lstm_step_forward(X[:, t, :], h, c, W, b)
    trace.append(c[shown, 0])
    if t < T - 1:
        hidden_peak = max(hidden_peak, np.abs(h).max())
```

- `rng.uniform(-1, 1, n)` — n random numbers spread evenly between -1 and 1.
- `rng.randn(n, 999) * 0.5` — normally distributed numbers with standard deviation 0.5.
- `np.zeros((n, T, 3))` — a 3-D array: trial, step, and which of the three inputs.
- `X[:, 0, 0], X[:, 0, 1] = v, 1.0` — step 0 of every trial: the value in input 0 and a 1 in the write flag.
- `X[:, 1:, 0] = noise[:, :T - 1]` — steps 1 onward get distractors in the value input.
- `X[:, -1, 2] = 1.0` — index -1 is the last step: raise the read flag there.
- `h, c, _ = lstm_step_forward(X[:, t, :], h, c, W, b)` — one step; the gates are not needed, so they go to `_`.
- `trace.append(c[shown, 0])` — remember one trial's cell state at every step, for the plot.
- `max(hidden_peak, np.abs(h).max())` — keep the largest hidden value seen so far.

### Scoring the result, and the RNN contrast

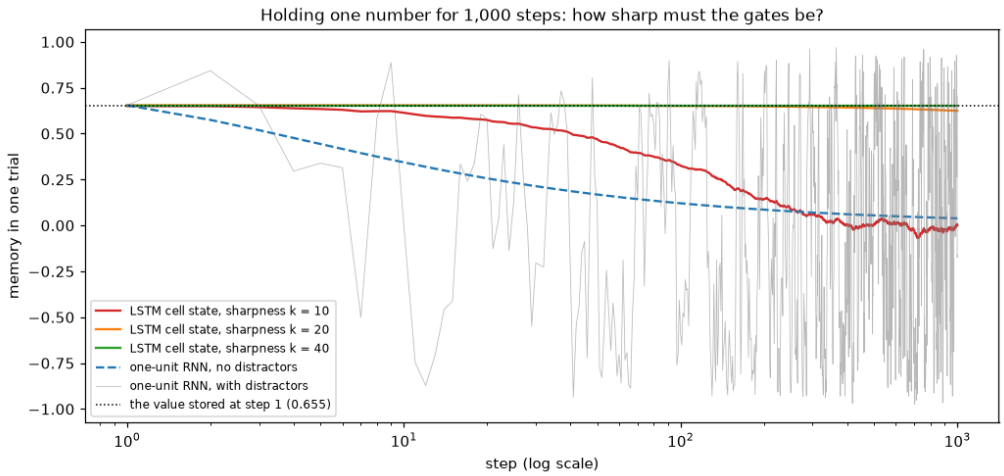
**CODE**

```
print(f"{k:>4}{sigmoid(k / 2):>13.9f}{sigmoid(-k / 2):>12.2e}{T - 1:>12}{np.mean(np.abs(c[:, 0] -
    stored)):>19.2e}"
      f"{np.corrcoef(c[:, 0], stored)[0, 1]:>13.6f}{hidden_peak:>21.2e}")
one = np.ones((1, 1))
h_noisy = rnn_step_forward(v[:, None], np.zeros((n, 1)), one, one, 0.0)
h_quiet = h_noisy.copy()
h_demo = np.tanh(np.array([[0.9]]))
tr_noisy, tr_quiet = [h_noisy[shown, 0]], [h_quiet[shown, 0]]
for t in range(T - 1):
    h_noisy = rnn_step_forward(noise[:, t:t + 1], h_noisy, one, one, 0.0)
    h_quiet = rnn_step_forward(np.zeros((n, 1)), h_quiet, one, one, 0.0)
    h_demo = np.tanh(h_demo)
```

- `np.mean(np.abs(c[:, 0] - stored))` — the average distance between what is in the cell and what was stored.
- `np.corrcoef(a, b)[0, 1]` — corrcoef returns a 2-by-2 table; the entry [0, 1] is the correlation between a and b.
- `{value:>19.2e}` — **e** format prints in powers of ten, such as 2.15e-01.
- `np.ones((1, 1))` — a 1-by-1 array holding 1.0, used as both of the RNN's weight matrices.
- `v[:, None]` — add an axis: (n,) becomes (n, 1), the shape rnn_step_forward expects.
- `h_noisy.copy()` — an independent copy; without .copy() both names would point at the same array.
- `noise[:, t:t + 1]` — column t kept 2-D, as (n, 1).

- `np.zeros((n, 1))` — an input of exactly zero: the no-distractor case.

**REAL GRAPH**




*Real graph from the script: one trial's memory over 1,000 steps, for the three sharpness settings and the one-unit RNN.*

# Segment 6 — demo_memory_survival

## Running a cell to the end, and the nudge measurement

**MATH**

$$\text{sensitivity} \approx \| \text{state}_T(s + \varepsilon d) - \text{state}_T(s - \varepsilon d) \| / 2\varepsilon \quad d = \text{a direction of length 1}$$

$$\text{LSTM direct route} = f_1 \cdot f_2 \cdot \ldots \cdot f_T \quad \text{(kept, in the code)}$$

**CODE**

```
def rnn_final_state(h0, X, Wxh, Whh):
    h = h0
    for t in range(X.shape[1]):
        h = rnn_step_forward(X[:, t:t + 1], h, Wxh, Whh, 0.0)
    return h
def lstm_final_cell(c0, X, W, b):
    h, c = np.zeros_like(c0), c0
    kept = np.ones_like(c0)
    for t in range(X.shape[1]):
        h, c, (f, _, _, _) = lstm_step_forward(X[:, t:t + 1], h, c, W, b)
        kept = kept * f
    return c, kept
def nudge_sensitivity(run, state0, direction, eps=1e-4):
    change = run(state0 + eps * direction) - run(state0 - eps * direction)
    return np.linalg.norm(change) / (2 * eps)
def fmt(value):
    return f"{value:.2e}" if value >= 1e-9 else "<1e-09"
```

- `for t in range(X.shape[1]):` — X.shape[1] is the number of steps.
- `rnn_step_forward(X[:, t:t + 1], h, Wxh, Whh, 0.0)` — the bias argument is given as 0.0: adding zero changes nothing.
- `np.zeros_like(c0), c0` — the hidden state starts at zero; the cell state starts at whatever was passed in.
- `np.ones_like(c0)` — an array of ones: the starting value for a running product.
- `h, c, (f, _, _, _) = ...` — unpack the gates tuple but keep only the forget gate.

- `kept = kept * f` — multiply in this step's forget gate; after the loop, kept is the product of all of them.
- `def nudge_sensitivity(run, state0, direction, eps=1e-4):` — **run** is a function passed in as an argument, so the same code measures the RNN and the LSTM.
- `run(state0 + eps * direction) - run(state0 - eps * direction)` — run twice, nudged up and nudged down, and take the difference.
- `np.linalg.norm(change) / (2 * eps)` — the size of the difference per unit of nudge.
- `f"{value:.2e}" if value >= 1e-9 else "<1e-09"` — a one-line if/else that returns text either way.

### The tool check and the two sweeps

**CODE**

```
direction = np.random.RandomState(1).randn(m, H)
direction /= np.linalg.norm(direction)
W[1:, :] = 0.0
X = np.random.RandomState(0).randn(m, 20)
kept = lstm_final_cell(zero, X, W, b)[1]
measured = nudge_sensitivity(lambda c0: lstm_final_cell(c0, X, W, b)[0], zero, direction)
exact = np.linalg.norm(kept * direction)
# inside the loop over T: the RNN sweep and the LSTM sweep
Whh *= rho / spectral_radius(Whh)
s = nudge_sensitivity(lambda h0: rnn_final_state(h0, X, Wxh, Whh), zero, direction)
rnn_curves[rho].append(s)
if T in table: table[T][("rnn", rho)] = s
W, b = init_lstm(1, H, seed=3, forget_bias=fb)
s = nudge_sensitivity(lambda c0: lstm_final_cell(c0, X, W, b)[0], zero, direction)
c_T, kept = lstm_final_cell(zero, X, W, b)
```

- `np.random.RandomState(1).randn(m, H)` — create a seeded generator and use it once, in one line.
- `direction /= np.linalg.norm(direction)` — divide in place so the whole array has length 1.
- `W[1:, :] = 0.0` — rows 1 onward are the weights from h; zeroing them cuts the gates off from the hidden state.
- `lstm_final_cell(zero, X, W, b)[1]` — call the function and take the second item it returns (the product of forget gates).
- `lambda c0: lstm_final_cell(c0, X, W, b)[0]` — an unnamed one-argument function. It remembers X, W and b from where it was written, so nudge_sensitivity only has to supply the state.
- `Whh *= rho / spectral_radius(Whh)` — rescale the matrix in place so its spectral radius becomes exactly rho.
- `if T in table: table[T][("rnn", rho)] = s` — a short if on one line; the inner key is a tuple of a label and a number.
- `init_lstm(1, H, seed=3, forget_bias=fb)` — same seed every time, so the only thing that differs between columns is the forget bias.

### Printing the tables

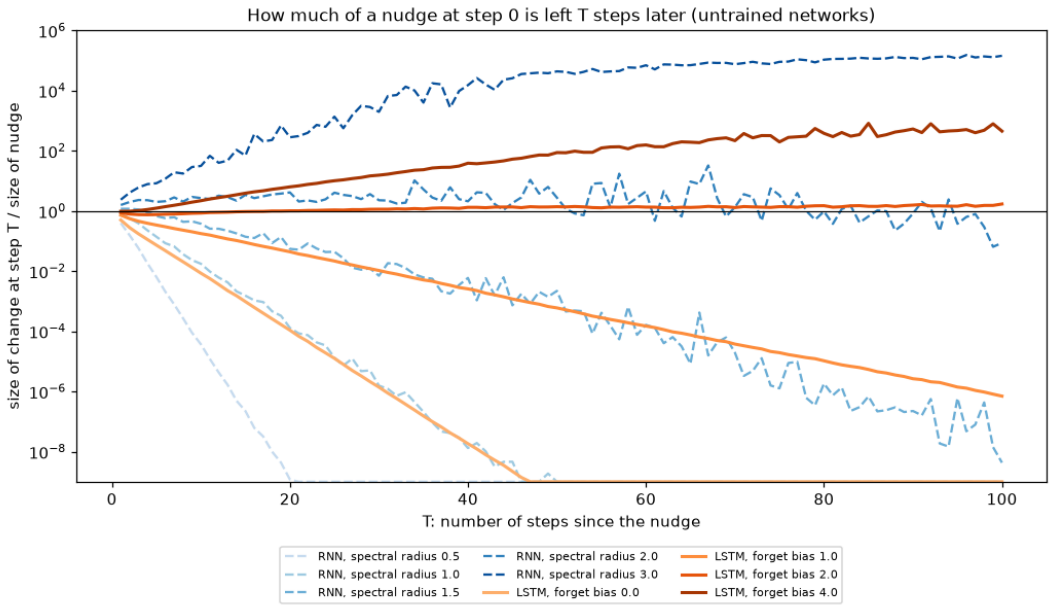
**CODE**

```
print(f"{'T':>5}" + "".join(f"{'rho=' + str(r):>12}" for r in rhos))
for T in lengths:
    print(f"{T:>5}" + "".join(f"{fmt(table[T][('rnn', r)]):>12}" for r in rhos))
```

- `"".join(... for r in rhos)` — build one piece of text per column and glue them together with nothing in between.
- `'rho=' + str(r)` — text joined to a number needs str() to turn the number into text first.
- `{...:>12}` — each column is right-aligned in 12 characters.

- `fmt(table[T][('rnn', r)])` — look the value up and let fmt decide between a number and the text <1e-09.

**REAL GRAPH**




*Real graph from the script: nudge sensitivity for every T from 1 to 100. np.clip keeps the curves from dropping below the 1e-9 floor of the method.*

# Segment 7 — demo_cost

## Counting weights and timing steps

**MATH**

$$\textit{RNN weights} = (D + H + 1) \cdot H \qquad \textit{LSTM weights} = 4 \cdot (D + H + 1) \cdot H$$

**CODE**

```
for D, H in [(1, 16), (32, 32), (64, 128)]:
    W, b = init_lstm(D, H, seed=0)
    n_rnn = D * H + H * H + H
    print(f"{D:>6}{H:>8}{n_rnn:>13}{W.size + b.size:>14}{(W.size + b.size) / n_rnn:>7.1f}")
D, H, m = 32, 32, 32
rng = np.random.RandomState(0)
x, h, c = rng.randn(m, D), rng.randn(m, H), rng.randn(m, H)
W, b = init_lstm(D, H, seed=0)
Wxh, Whh, bh = rng.randn(D, H), rng.randn(H, H), np.zeros(H)
start = time.perf_counter()
for _ in range(20000):
    lstm_step_forward(x, h, c, W, b)
t_lstm = time.perf_counter() - start
start = time.perf_counter()
for _ in range(20000):
    rnn_step_forward(x, h, Wxh, Whh, bh)
t_rnn = time.perf_counter() - start
```

- `for D, H in [(1, 16), (32, 32), (64, 128)]:` — each pair is unpacked into an input size and a hidden size.
- `W.size + b.size` — .size is the total number of entries in an array: the LSTM's weight count.

- `D * H + H * H + H` — the RNN's count: input weights, recurrent weights, biases.
- `time.perf_counter()` — a high-resolution clock reading, in seconds.
- `for _ in range(20000):` — repeat 20,000 times; the loop variable is not used, so it is named _.
- `time.perf_counter() - start` — seconds elapsed since the clock was read.

## Segment 8 — The cell diagram

### draw_cell_diagram

**CODE**

```
def box(x, y, w, h, title, sub, color):
    ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle="round,pad=0.04",
        facecolor=color, edgecolor="#333333", linewidth=1.2))
    ax.text(x, y + (0.17 if sub else 0.0), title, ha="center", va="center", fontsize=9.5,
        fontweight="bold")
    ax.text(x, y - 0.2, sub, ha="center", va="center", fontsize=8.5)
def op(x, y, symbol):
    ax.add_patch(Circle((x, y), 0.27, facecolor="white", edgecolor="#333333", linewidth=1.4))
    ax.text(x, y, symbol, ha="center", va="center", fontsize=13, fontweight="bold")
def arrow(x0, y0, x1, y1, color="#333333", width=1.5):
    ax.annotate("", xy=(x1, y1), xytext=(x0, y0), arrowprops=dict(arrowstyle="->", color=color,
        linewidth=width))
```

- `def box(...):` — a helper defined inside draw_cell_diagram, so it can use **ax** without being passed it.
- `FancyBboxPatch((x - w / 2, y - h / 2), w, h, ...)` — the shape wants its bottom-left corner, so half the width and height are subtracted from the centre.
- `y + (0.17 if sub else 0.0)` — an empty string counts as False: with no second line, the title sits in the middle of the box.
- `Circle((x, y), 0.27, ...)` — a circle centred on (x, y) with radius 0.27, for the multiply and add symbols.
- `ax.add_patch(...)` — put a shape on the drawing.
- `ax.annotate("", xy=..., xytext=..., arrowprops=dict(arrowstyle="->", ...))` — an arrow with no text, from xytext to xy.

## Quick Syntax Glossary — new or reinforced constructs

- `1.0 / (1.0 + np.exp(-z))` — the sigmoid; every gate in the week is made with it.
- `np.concatenate([a, b], axis=1)` — join arrays side by side; here the input and the old hidden state.
- `a[:, k * H:(k + 1) * H]` — the k-th block of H columns; how four gates share one matrix.
- `f * c + i * g` — entry-by-entry multiply and add: the whole cell-state update.
- `return h, c, (f, i, g, o)` — return a tuple inside a tuple; unpack with h, c, (f, i, g, o) = ...
- `lambda c0: fn(c0, X, W, b)[0]` — a function that remembers nearby variables, passed as an argument.
- `X[:, -1, 2] = 1.0` — index -1: the last position along that axis.
- `np.corrcoef(a, b)[0, 1]` — the correlation between two rows of numbers.
- `kept = kept * f` — a running product inside a loop.
- `np.clip(values, 1e-9, None)` — raise anything below 1e-9 up to 1e-9; None means no upper limit.
- `time.perf_counter()` — a precise clock for measuring how long code takes.

*End of Day 63 syntax reference. For the experiments and what they showed, see the Day 63 complete guide.*